Algorithm Analysis

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **identify** a loop hidden inside a method call.
2. Be able to **differentiate** code whose costs add from code whose costs multiply.
3. Be able to **justify** whether to sort before searching.

Recap: double N, and the steps...

Big-O	Double N, and the steps...
O(1)	stay the same
O(log N)	go up by one: one more halving
O(N)	double
$O(N^2)$	go up ×4
$O(N^3)$	go up ???

Tophat Q0

In $O(N^3)$, if N doubles, by what factor does the number of steps go up?

→ Open Tophat to vote.

Why does the size matter?

- Set up the answer key for a quiz? Q questions: **O(Q)**. The number of students doesn't matter.
- Grade the quiz? Every student, every question. **Two sizes!** S students × Q questions: **O(S × Q)**.

Which road-trip songs do I still need?

```
public static int countMissing(String[] roadTrip, String[] library) {
  int missing = 0;
  for (int i = 0; i < roadTrip.length; i++) {
    if (!contains(library, roadTrip[i])) {
      missing++;
    }
  }
  return missing;
}
```

One loop, over N road-trip songs. So O(N)? **Not quite...**

Which road-trip songs do I still need?

```
public static boolean contains(String[] library, String title) {
  for (int i = 0; i < library.length; i++) {
    if (library[i].equals(title)) {
      return true;
    }
  }
  return false;
}
```

`contains` loops over all M library songs (or less, if the song is there), **once per road-trip song**.

Eclipse Exercise

Get today's files from Canvas; run `CountMissing.java`

Two sizes

Road trip (M)	Library (N)	Steps
10	1,000	10,000
20	1,000	20,000
10	2,000	20,000

Either one doubles the steps: **O(M × N)**, like Monday's `countLetters`. This time, one loop was hidden in `contains`.

Add or multiply?

```
for (int i = 0; i < n; i++) { ... }   // N steps
for (int j = 0; j < n; j++) { ... }   // then N more

for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) { ... } // N steps, N times
}
```

One after another: **add**. N + N = 2N, so O(N).

One inside another: **multiply**. N × N, so O(N²).

Tophat Q1

What's the Big-O? M road-trip songs, N in the library.

```
for (int i = 0; i < roadTrip.length; i++) { ... }
for (int j = 0; j < library.length; j++) { ... }
```

→ Open Tophat to vote.

Keep the biggest term

A loop, then a nested loop, over the same N:

$N + N^2$ steps.

N	N^2	$N + N^2$
10	100	110
1,000	1,000,000	1,001,000

As N grows, the N^2 is nearly all of it.

$O(N + N^2) = O(N^2)$.

Two sizes, M + N? Keep both: either could be bigger.

Tophat Q2

What's the Big-O of this loop over N songs?

`totalSeconds` is Monday's method.

```
for (int i = 0; i < seconds.length; i++) {
  double percent = 100.0 * seconds[i] / totalSeconds(seconds);
  System.out.println(percent + "%");
}
```

→ Open Tophat to vote.

Searching an unsorted library

```
Q X F C A Z S U E
```

Is `A` here? Check each one until we find it: **O(N)**.

Binary search is Monday's dictionary trick: check the middle, keep one half. **O(log N)**...
but only on a **sorted** list.

Sort first?

```
Sort the library:   O(N log N)
Binary search:      O(log N)
Total:              O(N log N)
```

Sorting takes about N log N steps: more than N, far less than N^2.

For **one** search, that's worse than O(N). **Not worth it... yet.**

Eclipse Exercise

Open `DuplicatesSorted.java`. It copies the titles, sorts them so repeats sit side by side, then compares each one with the next.

Read the code: copy, sort, compare. Do the steps add, or multiply? Then run it.

Sorting first pays off here

1,000 songs	Steps
`hasDuplicateTitles`	499,500
`hasDuplicateTitlesSorted`	999, plus about 1,000 × 10 to sort

Copy, sort, compare: N + N log N + N, so **O(N log N)**, not O(N²). Unsorted, we search for each title's twin. Sorted, it's right next to it. **Many searches? Sort first.**

Tophat Q3

A quick check on when sorting first pays off.

→ Open Tophat to vote.

Questions?